



Name of the module: ATAN
Location in GIT repository: vspa_common\vspa-lib\atan

Overview:

This module provides kernels for full circle phase extraction corresponding to the `atan2()` function. The output phase will also be normalized to PI so that it can be computed in the same way and reported in *Single Precision*, *Half Precision* and *Half Fixed*. For a given complex value $z = real + i * imag$ the functions will return:

$$phase(z) = atan2(imag, real) / PI$$

where `atan2()` is the built-in Matlab function. This module is intended for general purpose use. One usage example would be the phase extraction of the L-STF or L-LTF autocorrelation for Frequency Offset Estimation.

Type of code:

- VCPU assembly functions
 - `atan2_x64_chf_hf_asm`
 - `atan2_x64_chf_sp_asm`
 - `atan2_x64_csp_hf_asm`
 - `atan2_x64_csp_sp_asm`
 - `atan2_x64_chp_hp_asm`
 - `atan2_x64_chp_sp_asm`
 - `atan2_x64_csp_hp_asm`

Assembly Functions API:

```
void atan2_x64_chf_hf_asm( cfixed16_t* inp_p,  
                          fixed16_t* out_p,  
                          uint32_t count);  
  
void atan2_x64_chf_sp_asm( cfixed16_t* inp_p,  
                          float32_t* out_p,  
                          uint32_t count);  
  
void atan2_x64_csp_hf_asm( cfloat32_t* inp_p,  
                          fixed16_t* out_p,  
                          uint32_t count);  
  
void atan2_x64_csp_sp_asm( cfloat32_t* inp_p,  
                          float32_t* out_p,  
                          uint32_t count);  
  
void atan2_x64_chp_hp_asm( cfloat16_t* inp_p,  
                          float16_t* out_p,  
                          uint32_t count);  
  
void atan2_x64_chp_sp_asm( cfloat6_t* inp_p,  
                          float32_t* out_p,  
                          uint32_t count);
```



```
void atan2_x64_csp_hp_asm( cfloat32_t* inp_p,
                          float16_t* out_p,
                          uint32_t count);
```

inp_p – Pointer of the input complex buffer (vector non-aligned).

out_p – Pointer of the output complex buffer (vector aligned).

count – Number of iterations, each processing 64 input samples and producing 64 output phases regardless of in/out type

Implementation plan

The full circle phase extraction can be performed using the *atan()* function:

- $phase(z) = atan(imag/real)$ if $(real \geq 0)$ (quadrant I and IV)
- $phase(z) = atan(imag/real) + \pi$ if $(real < 0)$ and $(imag \geq 0)$ (quadrant II)
- $phase(z) = atan(imag/real) - \pi$ if $(real < 0)$ and $(imag < 0)$ (quadrant III)

Hence the problem is reduced to finding the phase for quadrants I and IV. Finding the *atan(imag/real)* value with some approximation method will result in a low approximation error inside some range and a high error outside of that range. The main problem is that the input domain of the *atan()* function is the real axis $[-\infty, \infty]$. The *atan()* evaluation range can be reduced to a finite domain if we take into account:

- $atan(x) + atan(1/x) = \pi/2, x \geq 0$
- $atan(x) + atan(1/x) = -\pi/2, x < 0$

Therefore we can reduce the evaluation range to $x = [-1, 1]$ domain and mirror the computation outside of this domain using:

- $atan(x) = -\pi/2 - atan(1/x), x < -1$
- $atan(x) = \pi/2 - atan(1/x), x \geq 1$

Therefore we reduced the problem of computing *atan(x)* with $x = [-1, 1]$. Since the input domain is well defined, we will use the polynomial fitting method which is a polynomial based approximation with the polynomial coefficients chosen to minimize the squared error in a given input domain. As compared to the Taylor series approximation (keeping a number of terms from the Taylor expansion) where the approximation error grows fast and uncontrollably, the polynomial fitting method spreads the approximation error uniformly inside the fitting range. Since the *atan(x)* function is odd, the polynomial coefficients for $x^0, x^2, x^4, \dots$ are 0. Therefore the approximation will take the form:

$$atan_{approx}(x) = c_0 x^{2N-1} + c_1 x^{2N-3} + c_2 x^{2N-5} + \dots + c_{N-1} x^1$$

The N coefficients $c_0, c_1, \dots, c_{N-1}$ are generated using the *polyfit()* Matlab function where N is the number of effective (non-zero) coefficients.

Based on the above performance considerations and some implementation constraints, the following processing steps are identified:

1. Given $z = real + i * imag$ compute both $X = imag/real$ and $1/X = real/imag$
2. Compute the selection mask between X and $1/X$:
 - $mask_{inv_X} = mask(1 - |1/X|)$
3. Perform selection between X and $(-1/X)$:



- $X' = X + \text{mask_inv_X} * (-X) + \text{mask_inv_X} * (-1/X)$
- 4. Compute half circle phase correction term due to the mirroring procedure (normalized to $\text{PI}/2$):
 - $A_{\text{half}} = \text{sign}(X) * \text{mask_inv_X}$
- 5. Compute the full circle phase correction term due to the quadrant ambiguity (normalized to PI):
 - $A_{\text{full}} = -\text{mask}(\text{real}) * \text{sign}(\text{imag}) + \text{sign}(\text{imag})$
- 6. Compute total phase correction term (normalized to PI):
 - $A_{\text{total}} = A_{\text{half}} * 0.5 + A_{\text{full}}$
- 7. Compute X'^2 then $\text{atan}(X')$ and in the final step also add the total phase correction term:
 - $V = X'^2;$ (rmad)
 - $V = V * c0 + c1$ (maf)
 - $V = V * X'^2 + c2;$ (maf)
 -
 - $V = V * X' + A_{\text{total}}$ (maf)

In the above description the following notations were used:

- $\text{mask}(x) = 0$ for negative x , 1 otherwise
- $\text{sign}(x) = -1$ for negative x , 1 otherwise

The VSPA processor has hardware capabilities to generate such functions for full vectors with the following configurations:

- setB.creg 3,0; "normal mode": write V in VRA
- setB.creg 3,1; "sign mode" : write $\text{sign}(V) = +/-1$ in VRA
- setB.creg 3,2; "binary mode" : write $\text{mask}(V) = 0/1$ in VRA

Limitations

The kernels described above have the following limitations:

- The full circle phase extraction is performed on a multiple of 64 samples (regardless of the input/output precision) in order to benefit from the maximum VAU width processing capability which is 64 samples

DMEM requirements

Variable	Minimum size (half-words)	Minimum alignment (half-words)	Allocated by
inp_p	64	None	Caller
out_p	64	64	Caller

The input buffer is not required to be vector aligned whereas the output buffer must be vector aligned.

The kernels do not use scratch/persistent memory buffers.

The computation can also be performed in place (as long as the common in/out buffer is aligned).

Test plan:

- Tested features:



- Input length: multiple of 64 samples
 - Number of polynomial coefficients: from 3 to 10
 - Input/output precision: Half Fixed, Half Precision or Single Precision
 - Test corner cases input values: +/-0, +/-1 values for real/imag parts
- Take a look in **Atan_Testplan.xlsx** for test plan details
- Take a look in **Atan_Report.csv** for cycle count
- Take a look in **src/atan_coeff.h** for mean/max absolute error of the kernel approximation for each in/out precision and number of coefficients